DATHEON

SOFTWARE & DATA SOLUTIONS S.A.S.

DOCUMENTO DE ARQUITECTURA DE SOFTWARE

Sistema de Gestión Operativa Multisede

Bar de Moe · Inventario, toma de pedidos, facturación y reportería

Versión del documento	**1.0**
Fecha de emisión	29 de septiembre de 2026
Estado	Aprobado para la Sprint Review del Sprint 1
Cliente	Bar de Moe
Proveedor	DATHEON S.A.S.
Alcance cubierto	Sprint 1 · M1 Seguridad y M2 Parametrización
Repositorio	github.com/Juanse985/BAR-DE-MOE · rama `integracion/sprint-1`
Asignatura	Diseño de Algoritmos · UNIMINUTO

Equipo de trabajo

Angel De Jesús Paniza Sierra	Líder Técnico · Scrum Master
Juan Felipe Cortés Mendoza	Arquitecto
Juan David Tovío Montes	Analista de Requisitos
Juan Sebastián Rodríguez Jiménez	Líder de Calidad (QA)

Documento de carácter confidencial. Elaborado a partir del levantamiento de requisitos realizado con el cliente y del código fuente integrado del Sprint 1 (commit `040d62c`).

Control de versiones

Este documento se administra bajo control de versiones. Cada edición reemplaza a la anterior y solo la más reciente es válida para efectos de diseño y construcción.

Versión	Fecha	Descripción del cambio	Elaborado por
0.1	08/09/2026	Decisiones de arquitectura iniciales del esqueleto (ADR-01 a ADR-05), registradas en `docs/01-arquitectura.md`.	Juan Felipe Cortés
0.2	15/09/2026	Modelo de datos del M1 y del M2 implementado y probado en las ramas de trabajo.	Angel Paniza · Juan David Tovío
0.3	16/09/2026	Integración de las cuatro ramas. Se cierran los defectos DEF-01 a DEF-10 y se ajusta el control de acceso por sede.	Juan Sebastián Rodríguez
1.0	**29/09/2026**	**Primera versión formal del documento de arquitectura: introducción, diagramas de arquitectura, componentes, despliegue y modelo entidad-relación, buenas prácticas y diccionario de términos.**	**Equipo DATHEON**

Aprobaciones

Rol	Nombre	Responsabilidad sobre este documento
Arquitecto	Juan Felipe Cortés Mendoza	Autor de las decisiones de arquitectura y del despliegue
Líder Técnico	Angel De Jesús Paniza Sierra	Componentes del módulo de seguridad
Analista de Requisitos	Juan David Tovío Montes	Modelo entidad-relación y trazabilidad con el tablero del cliente
Líder de QA	Juan Sebastián Rodríguez Jiménez	Buenas prácticas, verificación y consolidación del documento

> **Dónde vive este documento.** El repositorio documental del proyecto está en GitHub, en la carpeta `docs/` del repositorio `BAR-DE-MOE`. Este PDF se publica además en la carpeta de *Ejecución* del aula, como lo pide la guía de la asignatura.

Tabla de contenido

1. Introducción 4
1.1 Propósito del documento 4
1.2 Alcance 4
1.3 A quién va dirigido 4
1.4 Documentos de referencia 5
2. Visión general de la solución 6
3. Diagrama de arquitectura 8
4. Diagrama de componentes 10
5. Diagrama de despliegue 12
6. Modelo entidad-relación 15
6.1 Base de seguridad (auth_db) 15
6.2 Base de parametrización (parametrizacion_db) 16
6.3 Reglas de integridad y relación entre bases 17
7. Decisiones de arquitectura 18
8. Requisitos no funcionales y cómo se verifican 20
9. Listado de buenas prácticas 22
10. Diccionario de términos 24
11. Anexos 27

Índice de figuras

Figura 1. Arquitectura general en capas 8
Figura 2. Diagrama de componentes 10
Figura 3. Diagrama de despliegue 12
Figura 4. Modelo entidad-relación · auth_db 15
Figura 5. Modelo entidad-relación · parametrizacion_db 16

1. Introducción

1.1 Propósito del documento

Este documento describe la arquitectura del **Sistema de Gestión Operativa Multisede** que DATHEON S.A.S. construye para el Bar de Moe. Su propósito es dejar por escrito, en un solo lugar, *cómo está armado el sistema y por qué*: qué piezas lo componen, dónde se despliega cada una, cómo se guardan los datos y qué reglas sigue el equipo para construirlo.

Sirve a tres fines concretos: orientar a quien entra al proyecto, sustentar ante el cliente que los requisitos del tablero de levantamiento están cubiertos, y dejar un punto de partida estable para los módulos de los sprints siguientes (inventario, pedidos, facturación y reportería).

1.2 Alcance

El sistema completo son seis módulos, comprometidos en cuatro sprints de dos semanas. Este documento describe la arquitectura de toda la solución, pero solo los dos primeros módulos están construidos:

Módulo	Qué cubre	Sprint	Estado
M1	Seguridad y gestión de usuarios: perfiles, contraseñas cifradas, bloqueo por reintentos, sesión única, cierre por inactividad y auditoría	Sprint 1	Construido y probado
M2	Parametrización maestra: sedes, mesas, tipos de producto, proveedores y catálogo de productos por sede	Sprint 1	Construido y probado
M3	Inventario independiente por sede	Sprint 2	Pendiente
M4	Toma de pedidos y gestión de mesas	Sprint 2	Pendiente
M5	Facturación y medios de pago	Sprint 3	Pendiente
M6	Reportería con exportación a Excel	Sprint 4	Pendiente

Queda fuera del alcance todo lo excluido en la propuesta comercial: domicilios, cover, propinas, reservas, traslado de inventario entre sedes, cierre de caja, nómina, CRM, integración con el sistema legado y facturación electrónica ante la DIAN.

1.3 A quién va dirigido

- **Al equipo de desarrollo:** para saber dónde va cada cosa y no duplicar trabajo.
- **Al cliente y al Product Owner:** para ver, en lenguaje claro, cómo se cumplen los requisitos que pidió.
- **Al docente y a los evaluadores:** para sustentar las decisiones técnicas del proyecto.
- **A quien mantenga el sistema después:** para entender el porqué de cada decisión sin tener que preguntar.

1.4 Documentos de referencia

Documento	Contenido
Propuesta comercial v2 (PC-001-2026)	Alcance contratado, exclusiones, plan de sprints e inversión
Tablero de levantamiento de requisitos	Requisitos del cliente: 32 historias de usuario y 12 requisitos no funcionales
`docs/01-arquitectura.md`	Decisiones de arquitectura (ADR) tomadas al inicio del Sprint 1
`docs/02-plan-ramas-sprint-1.md`	Reparto del trabajo por rama y Definition of Done
`docs/03-convenciones-git.md`	Convenciones de ramas, commits y Pull Requests
`docs/04-guia-rapida.md`	Cómo levantar y probar la plataforma
`docs/05-contratos.md`	Mapeo campo por campo entre el tablero, la pantalla, la API y la tabla
`docs/06-owasp.md`	Los cuatro controles de seguridad comprometidos y su evidencia
`docs/07-informe-qa-sprint-1.md`	Defectos encontrados y riesgos de integración
`docs/08-integracion-sprint-1.md`	Cómo se unieron las cuatro ramas y qué aportó cada integrante

2. Visión general de la solución

El Bar de Moe opera varias sedes con registros manuales y un sistema legado que no cubre el ciclo completo del negocio. Eso produce descuadres de inventario, falta de trazabilidad sobre quién registra cada movimiento y ausencia de información consolidada para decidir.

La solución es una **aplicación web responsive** construida como un conjunto de **microservicios**: piezas pequeñas e independientes, cada una dueña de sus datos, que se comunican por HTTP y se publican al mundo a través de una sola puerta de entrada.

Por qué microservicios y no una sola aplicación

Razón	Qué significa en este proyecto
El alcance crece por módulos	M3 a M6 se agregan como servicios nuevos sin tocar lo que ya funciona y está aceptado por el cliente
Cuatro personas en paralelo	Cada integrante trabaja en su carpeta; los choques de código se reducen al mínimo
Aislamiento de fallas	Si el servicio de reportes se cae, el bar sigue tomando pedidos y facturando
Seguridad concentrada	La sesión se valida en un solo punto: el gateway. Los servicios no se exponen a la red

El costo que se acepta a conciencia: no hay *joins* entre bases de datos y hay más piezas que operar. Para el tamaño de este sistema, la ventaja de poder crecer por módulos pesa más.

Perfiles de usuario

Perfil	Qué puede hacer	Alcance de datos
Administrador	Parametrización completa, gestión de usuarios, restablecimiento de contraseñas, consulta de auditoría y de reportes	Todas las sedes
Cajero	Cierra pedidos, factura, registra el medio de pago y consulta el reporte de ventas	Solo su sede
Mesero	Consulta el mapa de mesas, verifica existencias y registra pedidos	Solo su sede

Tecnologías

Pieza	Tecnología	Por qué
Servicios y gateway	Python 3.12 · FastAPI · Uvicorn	Documentación automática de la API (Swagger), validación de datos incorporada y buen desempeño
Acceso a datos	SQLAlchemy 2.0	Consultas parametrizadas: cierra la puerta a la inyección SQL
Validación de datos	Pydantic 2	

Pieza	Tecnología	Por qué
		Tipo y longitud de cada campo se validan antes de llegar a la base
Base de datos	PostgreSQL 16	Transaccional, maduro y gratuito; una base por servicio
Autenticación	JWT (PyJWT) · bcrypt	Token firmado sin estado en el cliente; contraseñas con hash de 12 rondas
Frontend	HTML, CSS y JavaScript sin framework	Liviano, se sirve desde el mismo gateway y funciona en tablet y celular
Empaquetado	Docker y Docker Compose	La misma plataforma se levanta igual en cualquier computador
Calidad	pytest · pytest-cov · ruff · GitHub Actions	292 pruebas automáticas y cobertura mínima del 70 % antes de integrar

3. Diagrama de arquitectura

La arquitectura se organiza en cuatro capas. Cada flecha del diagrama es una comunicación real por HTTP; no hay atajos entre capas ni servicios que lean la base de datos de otro.

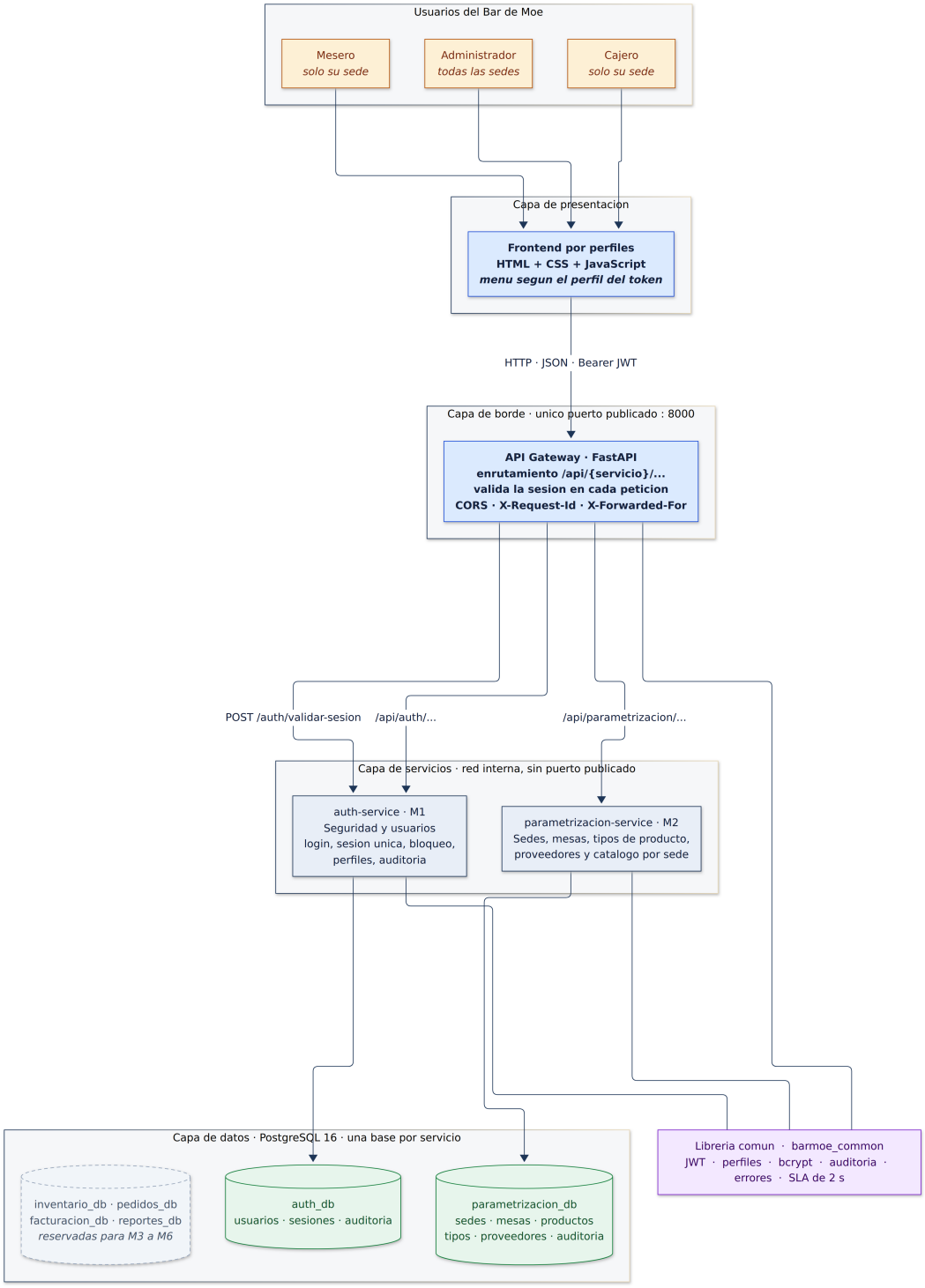


Figura 1. Arquitectura general en capas del Sistema de Gestión Operativa Multisede.

Qué hace cada capa

Capa	Responsabilidad	Tecnología
Presentación	Muestra a cada usuario solo lo que le corresponde por su perfil, valida los datos antes de enviarlos y cierra la sesión sola a los 3 minutos de inactividad	HTML, CSS y JavaScript
Borde (gateway)	Único punto expuesto. Enruta las peticiones, verifica que la sesión siga viva, aplica CORS, agrega el identificador de rastreo y la IP real del cliente	FastAPI
Servicios	Contienen las reglas del negocio. Cada uno valida la firma del token y el perfil del usuario antes de ejecutar cualquier operación	FastAPI + SQLAlchemy
Datos	Guarda la información. Una base por servicio, sin tablas compartidas	PostgreSQL 16

El camino de una petición, paso a paso

1. El mesero abre el mapa de mesas en su tablet. El navegador envía `GET /api/parametrizacion/mesas` con su token.
2. El **gateway** recibe la petición, genera un `X-Request-Id` y le pregunta al **auth-service** si esa sesión sigue viva.
3. El auth-service revisa que la sesión esté activa, que el usuario siga habilitado y que no lleve más de 3 minutos quieto. De paso, refresca el contador de inactividad.
4. El gateway reenvía la petición al **parametrizacion-service**, con la IP real del cliente y el identificador de rastreo.
5. El parametrizacion-service vuelve a validar la firma del token, ve que el perfil es MESERO y **fuerza el filtro a la sede del token**: aunque pida otra sede, recibe un rechazo.
6. La respuesta regresa con la cabecera `X-Tiempo-Ms`, que deja registrado cuánto tardó la operación.

> **Dos validaciones distintas, a propósito.** La *firma* del token se valida en cada servicio (defensa en profundidad: ningún servicio confía en que otro ya lo hizo). El *estado* de la sesión —viva, inactiva o única— se valida en el gateway, porque ese estado vive en una sola base y replicarlo sería duplicar la verdad.

4. Diagrama de componentes

El diagrama muestra las piezas internas de cada parte del sistema y cómo se usan entre ellas. Las flechas punteadas indican dependencia de la librería común.

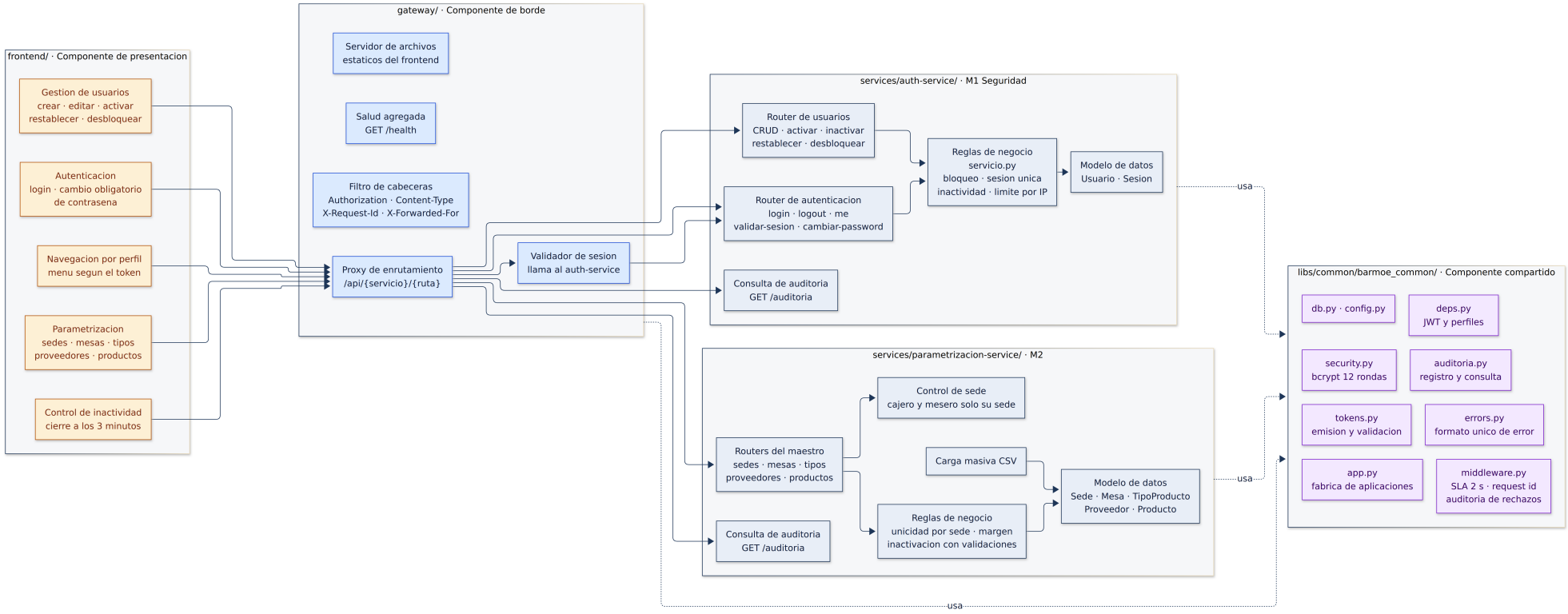


Figura 2. Componentes internos del sistema y sus dependencias.

Responsabilidad de cada componente

Componente	Ubicación	Responsabilidad
Frontend por perfiles	`frontend/`	Login, cambio obligatorio de contraseña, menú por perfil, pantallas de usuarios y de parametrización, y cierre por inactividad
Proxy de enrutamiento	`gateway/app/main.py`	Traduce `/api/{servicio}/{ruta}` a la dirección interna del servicio
Validador de sesión	`gateway/app/main.py`	Consulta al auth-service en cada petición protegida; hace cumplir la inactividad y la sesión única
Filtro de cabeceras	`gateway/app/main.py`	Solo deja pasar `Authorization`, `Content-Type` y `Accept`; agrega el request id y la IP real
Router de autenticación	`auth-service/app/main.py`	Login, logout, datos del usuario en sesión, validación de sesión y cambio de contraseña
Router de usuarios	`auth-service/app/main.py`	Crear, consultar, editar, activar, inactivar, restablecer contraseña y desbloquear
Reglas de seguridad	`auth-service/app/servicio.py`	Bloqueo por reintentos, límite por IP, sesión única, ventana de inactividad y registro en auditoría
Routers del maestro	`parametrizacion-service/app/main.py`	Sedes, mesas, tipos de producto, proveedores y catálogo de productos por sede
Control de sede	`parametrizacion-service/app/main.py`	Obliga a cajeros y meseros a ver únicamente los datos de su sede
Carga masiva CSV	`parametrizacion-service/app/main.py`	Carga el catálogo inicial de productos; una fila con error no tumba las demás
Consulta de auditoría	`libs/common/.../auditoria.py`	Router de solo lectura con filtros y paginación, disponible en los dos servicios
Seguridad de contraseñas	`libs/common/.../security.py`	Hash bcrypt de 12 rondas y reglas mínimas de la contraseña
Tokens y permisos	`libs/common/.../tokens.py`, `deps.py`	Emisión y validación del JWT; dependencia que exige un perfil determinado
Middlewares	`libs/common/.../middleware.py`	Mide el tiempo de cada respuesta, propaga el request id y audita los accesos rechazados
Formato de errores	`libs/common/.../errors.py`	Todos los errores salen con la misma forma: código legible y mensaje

5. Diagrama de despliegue

El diagrama muestra en qué máquina y en qué contenedor corre cada pieza, qué puertos se abren y por dónde viajan los datos.

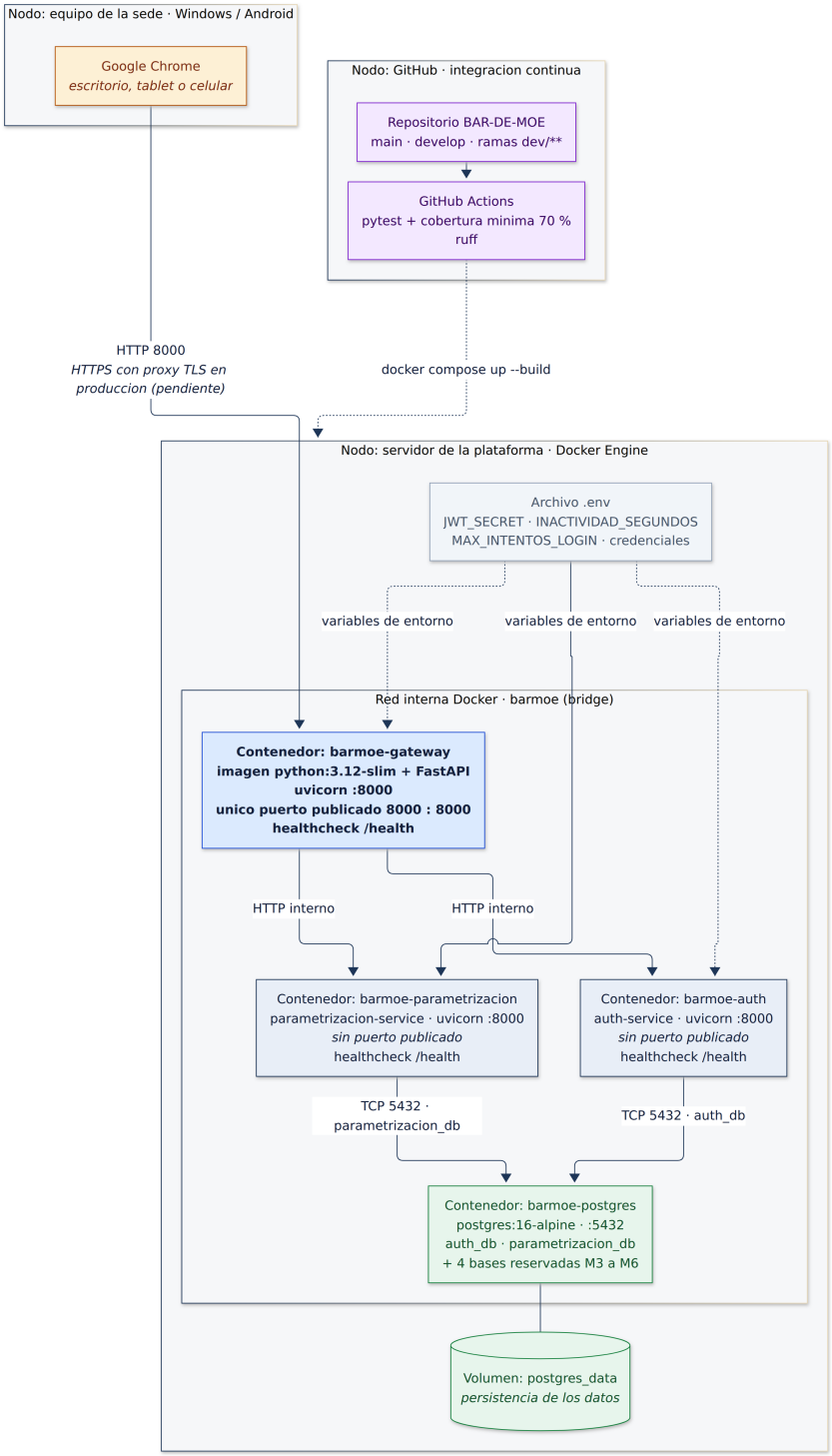


Figura 3. Despliegue de la plataforma con Docker Compose.

Nodos y contenedores

Contenedor	Imagen base	Puerto	Notas
`barmoe-gateway`	python:3.12-slim	**8000 publicado**	Única puerta de entrada. También sirve el frontend
`barmoe-auth`	python:3.12-slim	8000 interno	No se publica al host (ADR-02)
`barmoe-parametrizacion`	python:3.12-slim	8000 interno	No se publica al host
`barmoe-postgres`	postgres:16-alpine	5432	Publicado solo para inspeccionar la base en desarrollo

Los tres servicios traen *healthcheck*: Docker no da por arrancado un contenedor hasta que su `/health` responde. El gateway espera a que auth y parametrización estén sanos.

Configuración por variables de entorno

Ninguna credencial vive en el código. Todo llega por el archivo `.env`, que no se sube al repositorio:

Variable	Valor por defecto	Para qué
`JWT_SECRET`	—	Firma de los tokens. En producción se genera con `openssl rand -hex 32`
`INACTIVIDAD_SEGUNDOS`	180	RNF-03: cierre de sesión a los 3 minutos
`MAX_INTENTOS_LOGIN`	3	RNF-09: bloqueo por reintentos fallidos
`PERMITIR_MULTISESION`	false	RNF-10: sesión única
`BLOQUEO_MINUTOS`	0	0 = bloqueado hasta que el administrador desbloquee
`MAX_INTENTOS_POR_IP` / `VENTANA_IP_MINUTOS`	10 / 15	Frena el ataque que prueba una contraseña contra muchos usuarios
`ORIGENES_PERMITIDOS`	localhost	CORS: desde qué direcciones se acepta el navegador
`PROXIES_CONFIABLES`	vacío	IP del proxy con TLS cuando exista, para respetar su cabecera de IP real

Ambientes

Ambiente	Cómo se levanta	Base de datos
Desarrollo local	`docker compose up --build`, o los tres servicios con `uvicorn`	PostgreSQL en contenedor, o SQLite para programar rápido
Pruebas automáticas	`make cov` · también en GitHub Actions en cada push	SQLite en archivos temporales
Demostración (Sprint Review)		PostgreSQL con el volumen `postgres_data`

Ambiente	Cómo se levanta	Base de datos
	`docker compose up` + `make evidencias`	
Producción en la sede piloto	Pendiente: se agrega un proxy inverso con TLS delante del gateway	PostgreSQL con respaldo programado

6. Modelo entidad-relación

Por la decisión ADR-01 (una base por servicio) el modelo se presenta en dos partes, una por base de datos. Las dos comparten la misma estructura de auditoría.

6.1 Base de seguridad · `auth_db`

Guarda quién puede entrar al sistema, con qué perfil y desde qué sesión. Los siete campos que pidió el cliente para el usuario —cédula, nombre, sede, perfil, usuario, contraseña y estado— están en la tabla `usuarios`. Las demás columnas existen para sostener reglas del tablero, no para capturar datos nuevos del cliente.

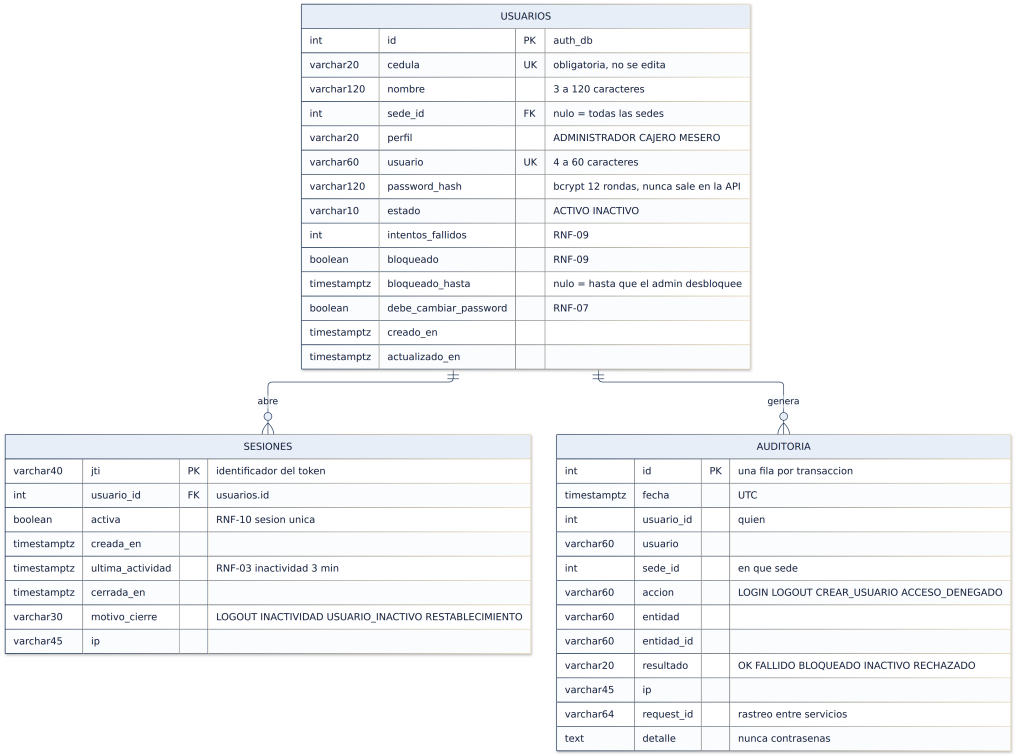


Figura 4. Modelo entidad-relación de `auth_db` (M1 Seguridad).

6.2 Base de parametrización · `parametrizacion_db`

Guarda la configuración del bar: sus sedes, las mesas de cada sede, los tipos de producto, los proveedores y el catálogo de productos. El catálogo se gestiona **por sede**: el mismo código de producto puede existir en varias sedes, pero no dos veces en la misma.

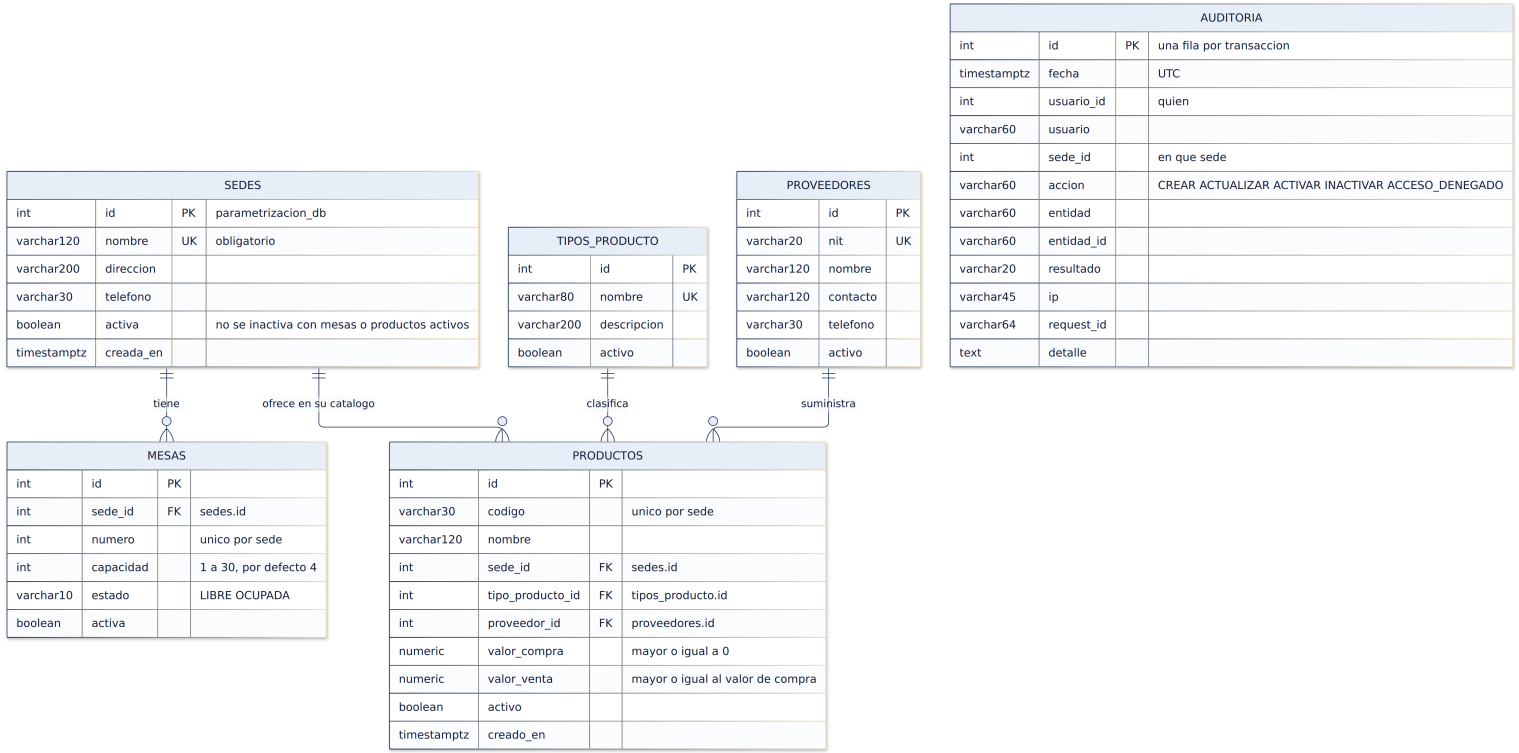


Figura 5. Modelo entidad-relación de `parametrizacion_db` (M2 Parametrización).

6.3 Reglas de integridad y relación entre bases

Regla	Dónde se aplica
Cédula y nombre de usuario únicos	Restricción `UNIQUE` en `usuarios`
Una mesa no se repite en la misma sede	Restricción `uq_mesa_por_sede (sede_id, numero)`
Un código de producto no se repite en la misma sede	Restricción `uq_producto_por_sede (sede_id, codigo)`
El valor de venta nunca es menor que el de compra	Validación en el contrato de entrada y al editar
No se borra nada: se inactiva	Columnas `activa` / `activo` / `estado`. El historial se conserva
Una sede no se inactiva si tiene mesas o productos activos	Validación en el servicio de parametrización
Los registros de auditoría no se modifican ni se borran	Solo existe la operación de consulta (`GET /auditoria`)

Relación entre las dos bases. El campo `usuarios.sede_id` apunta a `sedes.id`, pero como están en bases distintas **no hay llave foránea**: es una referencia lógica. Hoy nada impide crear un cajero en una sede que no existe, y ese usuario vería listas vacías. Está registrado como hallazgo H-05 en `docs/05-contratos.md` y la solución propuesta es validar la sede por HTTP contra el servicio de parametrización al crear el usuario.

7. Decisiones de arquitectura

Cada decisión se registró cuando se tomó, con su motivo y su costo. Así nadie tiene que adivinar después por qué el sistema es como es.

ID	Decisión	Por qué y qué costo tiene
ADR-01	Una base de datos por microservicio	Evita que un cambio de esquema rompa a los demás servicios. Costo aceptado: no hay *joins* entre servicios; cuando el reporte del M6 necesite cruzar datos, se compondrá en el servicio de reportes
ADR-02	El gateway es el único punto expuesto	Un solo lugar donde se valida la sesión y se aplica CORS, y ningún atacante llega directo a un microservicio
ADR-03	La firma del token se valida en cada servicio; el estado de la sesión, en el gateway	Defensa en profundidad sin duplicar el estado de la sesión en varias bases
ADR-04	Sesión única por rechazo, no por expulsión	Si dos personas usan la misma cuenta, el sistema lo hace evidente en vez de resolverlo en silencio. Se puede cambiar sin tocar código con `PERMITIR_MULTISESION=true`
ADR-05	Los requisitos no funcionales viven en el código, no solo en un documento	Cada RNF tiene una prueba automática que lo verifica; si alguien lo rompe, el CI se pone en rojo
ADR-06	El código habla el idioma del negocio (español)	El tablero del cliente, la pantalla, la API y las tablas usan los mismos nombres: es más fácil de revisar y de sustentar
ADR-07	Una librería común para lo transversal	Seguridad, auditoría, errores y medición del tiempo se escriben una sola vez. Costo: es código compartido, así que se avisa al equipo antes de modificarlo

Deuda técnica reconocida

Se declara de frente para que nadie se sorprenda en la sustentación:

Pendiente	Riesgo	Cuándo
Sin migraciones (Alembic)	Las tablas se crean al arrancar; un cambio de columna deja las bases del equipo desalineadas	Sprint 2, prioridad alta
Sin HTTPS	En desarrollo el tráfico va en claro; para la sede piloto se necesita un proxy con TLS	Antes de producción
Contraseña inicial no obligatoria en el servidor	Hoy solo el frontend obliga a cambiarla	Sprint 2
Sin límite de peticiones en el gateway	El límite por IP hoy vive en el servicio de seguridad, no en el borde	Sprint 2
Sin refresh token	Al expirar el token, el usuario vuelve a autenticarse	Por evaluar con el cliente

8. Requisitos no funcionales y cómo se verifican

Cada requisito transversal del tablero tiene un lugar en el código y una forma de comprobarse. No basta con estar escrito: si no se puede demostrar en la Sprint Review, no se da por cumplido.

ID	Requisito	Dónde está implementado	Cómo se comprueba
RNF-01	Diseño responsive	Hojas de estilo del frontend	Revisión en navegador a 390 px: sin desplazamiento horizontal
RNF-02	Respuesta en 2 segundos	`TiempoRespuestaMiddleware`	Cabecera `X-Tiempo-Ms` en toda respuesta; medición del percentil 95 bajo carga
RNF-03	Cierre por inactividad de 3 minutos	`sesiones.ultima_actividad` + gateway	Pruebas con 179 y 181 segundos de inactividad
RNF-04	Compatibilidad con Google Chrome	Frontend sin dependencias exóticas	Revisión de las pantallas en Chrome
RNF-05	Tres perfiles de acceso	`deps.requiere_perfil`	Pruebas negativas: cada operación se intenta con un perfil no autorizado
RNF-06	N usuarios, N sedes y N mesas	Modelo de datos sin límites fijos	Listados paginados; el catálogo se carga por CSV
RNF-07	Cambio de contraseña por el usuario y por el administrador	`cambiar_password` · `restablecer_password`	Una prueba por cada vía
RNF-08	Contraseñas cifradas	`security.hashear_password` (bcrypt, 12 rondas)	Se recorren todas las tablas: ninguna contraseña legible
RNF-09	Bloqueo por reintentos	`servicio.login`	Tres intentos fallidos bloquean; ni la contraseña correcta entra después
RNF-10	Sesión única	`servicio.login`	El segundo login se rechaza y la

ID	Requisito	Dónde está implementado	Cómo se comprueba
			primera sesión sigue viva
RNF-11	Seguridad verificable (OWASP)	Ver sección 9 y `docs/06-owasp.md`	Cuatro controles con prueba propia
RNF-12	Trazabilidad de las transacciones	`auditoria.registrar`	Cada escritura deja usuario, sede, fecha, IP y request id

Resultado de la verificación del Sprint 1

Verificación	Resultado
Pruebas automáticas (4 componentes)	292 en verde, 0 fallidas, 0 defectos abiertos
Cobertura de código	Entre 94 % y 99 % por componente (umbral acordado: 70 %)
Prueba de humo de punta a punta	22 de 22 pasos
Tiempo de respuesta (RNF-02)	Cumple: la operación más lenta es el login, con unos 0,3 s
Revisión del frontend con los tres perfiles	12 de 12 verificaciones
Colección Postman	27 peticiones, 25 aserciones, sin fallos

9. Listado de buenas prácticas

Estas son las prácticas que el equipo acordó y aplica. No son teoría: cada una se puede señalar en el repositorio.

9.1 Arquitectura y diseño

1. **Una base de datos por servicio.** Ningún servicio lee las tablas de otro; se hablan por HTTP.
2. **Un solo punto de entrada.** Los servicios internos no publican puertos al exterior.
3. **Decisiones registradas (ADR).** Cada decisión se escribe con su motivo y su costo.
4. **Configuración por variables de entorno.** El mismo código sirve para desarrollo y producción.
5. **Lo transversal se escribe una sola vez** en una librería común.
6. **Nombres del negocio en el código.** Tablero, pantalla, API y tablas usan las mismas palabras.

9.2 Seguridad

1. **Contraseñas con hash bcrypt de 12 rondas** y sal incluida. Nunca se guardan ni se devuelven en claro.
2. **Permisos validados en el servidor,** no solo escondiendo botones en la pantalla.
3. **El perfil y la sede se verifican en cada operación,** no solo al iniciar sesión.
4. **Consultas parametrizadas** con el ORM: no se arma SQL concatenando texto.
5. **Validación de tipo y longitud** de cada campo antes de tocar la base de datos.
6. **Mensajes de error que no revelan de más:** el login no dice si el usuario existe.
7. **Bloqueo por reintentos y límite por IP** contra los ataques de fuerza bruta.
8. **Sesión única y cierre por inactividad,** pensados para un equipo compartido en la barra.
9. **Todo intento rechazado queda auditado** con usuario, sede, IP y hora.
10. **Ninguna credencial en el repositorio:** el archivo `.env` está en `.gitignore`.
11. **Cabeceras filtradas en el gateway:** el navegador no puede inyectar perfil ni sede.

9.3 Datos

1. **Inactivación lógica en lugar de borrado:** el historial del bar se conserva.
2. **Restricciones de unicidad en la base,** no solo en el código.
3. **Valores de dinero en `NUMERIC`,** nunca en coma flotante.
4. **Fechas en UTC** con zona horaria, para que la auditoría no dependa del computador.
5. **Auditoría de solo lectura:** no existe forma de modificar o borrar el registro desde la aplicación.

9.4 Código y API

1. **Un formato único de error** para toda la plataforma: código legible y mensaje.
2. **Rutas y verbos coherentes,** con documentación automática en Swagger.

3. **Listados paginados** con el total en la cabecera `X-Total-Count`.
4. **Estilo revisado automáticamente** con `ruff` en cada push.
5. **Las reglas de negocio viven en el servicio,** no en las pantallas ni en la base.

9.5 Pruebas y calidad

1. **Las pruebas llevan el requisito en el nombre** (`test_rnf09_...`): la evidencia sale del propio reporte.
2. **Cobertura mínima del 70 %,** exigida por el CI.
3. **Pruebas negativas por perfil:** se comprueba lo que cada usuario *no* puede hacer.
4. **Prueba de humo de punta a punta** contra la plataforma levantada de verdad.
5. **Medición del desempeño** con percentil 95, no solo con el promedio.
6. **Los defectos conocidos quedan como prueba marcada,** así se avisa solo cuando alguien los corrige.

9.6 Trabajo en equipo y operación

1. **Una rama por integrante** y todo lo que entra pasa por Pull Request.
2. **Commits con formato acordado** (*Conventional Commits*, en español).
3. **Nadie aprueba su propio Pull Request** ni integra con el CI en rojo.
4. **Definition of Done escrita:** pruebas en verde, endpoint documentado, operación auditada y respuesta bajo 2 segundos.
5. **Avisar antes de tocar la librería común,** porque afecta a todos.
6. **Datos de ejemplo reproducibles** con un solo comando, para que la demo nunca dependa de la suerte.
7. **Estado de salud expuesto** en `/health`, con el detalle de cada servicio.

10. Diccionario de términos

API
Conjunto de operaciones que un programa ofrece para que otro lo use. Aquí, las rutas que el frontend llama para leer y guardar datos.

ADR (Architecture Decision Record)
Registro corto de una decisión de arquitectura: qué se decidió, por qué y qué costo tiene.

Auditoría
Tabla donde queda constancia de cada operación: quién la hizo, en qué sede, cuándo, desde qué IP y con qué resultado.

bcrypt
Algoritmo para guardar contraseñas. Produce un valor del que no se puede volver a la contraseña original. Las "rondas" definen cuánto cuesta calcularlo: más rondas, más difícil de atacar.

CI (Integración continua)
Servicio que, en cada cambio subido al repositorio, corre las pruebas y el revisor de estilo de forma automática.

Contenedor
Paquete que lleva un programa junto con todo lo que necesita para funcionar. Corre igual en cualquier computador con Docker.

CORS
Regla del navegador que define desde qué direcciones web se puede llamar a la API.

Cobertura de código
Porcentaje del código que las pruebas automáticas llegan a ejecutar.

Docker Compose
Archivo que describe todos los contenedores del sistema y los levanta juntos con un solo comando.

Endpoint
Cada dirección concreta de la API, por ejemplo `POST /api/auth/auth/login`.

Gateway
Servicio que recibe todas las peticiones del navegador y las reparte a los servicios internos. Es la única puerta abierta del sistema.

Hash
Resultado de transformar un dato en un valor de tamaño fijo del que no se puede regresar al original.

Healthcheck
Revisión automática que hace Docker para saber si un contenedor está sano.

HU (Historia de usuario)
Requisito escrito desde la necesidad de quien lo usa: "yo como mesero quiero…, para…".

JWT (JSON Web Token)

Credencial firmada que el sistema entrega al iniciar sesión. Lleva dentro el usuario, su perfil y su sede, y viaja en cada petición.

Microservicio
Servicio pequeño e independiente, dueño de sus datos, que cumple una parte del sistema y se comunica con los demás por HTTP.

Middleware
Código que se ejecuta antes o después de cada petición. Aquí mide el tiempo de respuesta y registra los accesos rechazados.

Migración
Cambio versionado del esquema de la base de datos, para que todos los ambientes queden iguales. Está pendiente (Alembic).

ORM
Librería que traduce entre objetos del programa y tablas de la base de datos. Aquí es SQLAlchemy, y genera consultas parametrizadas.

OWASP
Organización que publica la lista de los riesgos de seguridad más comunes en aplicaciones web. El proyecto se compromete con cuatro de ellos.

Percentil 95 (p95)
Valor por debajo del cual está el 95 % de las mediciones. Se usa para el tiempo de respuesta porque muestra la experiencia real, no solo el promedio.

Perfil
Rol del usuario dentro del sistema: administrador, cajero o mesero. Define qué puede hacer y sobre qué datos.

Pull Request
Propuesta de incorporar los cambios de una rama, que otro integrante revisa y aprueba antes de integrarla.

RNF (Requisito no funcional)
Requisito sobre cómo debe comportarse el sistema —velocidad, seguridad, disponibilidad—, no sobre qué hace.

Request id
Identificador único que acompaña a una petición por todos los servicios, para poder seguirle el rastro.

Responsive
Diseño que se acomoda al tamaño de la pantalla: computador, tablet o celular.

Sede
Cada local del Bar de Moe. Tiene sus propias mesas, su catálogo y su inventario.

Sesión única
Regla que impide que el mismo usuario tenga dos sesiones abiertas al tiempo.

SLA
Compromiso de servicio. Aquí, que toda transacción responda en 2 segundos o menos.

Sprint
Periodo de dos semanas al final del cual se entrega software funcionando y el cliente lo acepta o lo rechaza.

Swagger
Página que FastAPI genera sola con la documentación de la API y un formulario para probarla.

Token
Ver JWT.

Trazabilidad
Poder reconstruir quién hizo qué, dónde y cuándo. La sostiene la tabla de auditoría.

11. Anexos

11.1 Principales operaciones de la API

Todas pasan por el gateway, en `http://localhost:8000`.

Operación	Perfil	Para qué
`POST /api/auth/auth/login`	Todos	Iniciar sesión y recibir el token
`POST /api/auth/auth/logout`	Todos	Cerrar la sesión
`POST /api/auth/auth/cambiar-password`	Todos	Cambiar la propia contraseña
`GET /api/auth/usuarios`	Administrador	Listado paginado con filtros por sede, perfil y estado
`POST /api/auth/usuarios`	Administrador	Crear un usuario
`POST /api/auth/usuarios/{id}/inactivar` · `/activar`	Administrador	Habilitar o deshabilitar una cuenta sin borrar su historial
`POST /api/auth/usuarios/{id}/restablecer-password`	Administrador	Devolver el acceso a quien olvidó la contraseña
`POST /api/auth/usuarios/{id}/desbloquear`	Administrador	Levantar el bloqueo por reintentos
`GET /api/auth/auditoria`	Administrador	Consultar la auditoría de seguridad
`GET` · `POST` · `PATCH /api/parametrizacion/sedes`	Consulta: todos · Escritura: administrador	Administrar las sedes
`GET` · `POST` · `PATCH /api/parametrizacion/mesas`	ídem	Administrar las mesas de cada sede
`GET` · `POST` · `PATCH /api/parametrizacion/tipos-producto`	ídem	Administrar los tipos de producto
`GET` · `POST` · `PATCH /api/parametrizacion/proveedores`	ídem	Administrar los proveedores
`GET` · `POST` · `PATCH /api/parametrizacion/productos`	ídem	Catálogo de productos por sede

Operación	Perfil	Para qué
`POST /api/parametrizacion/productos/carga-masiva`	Administrador	Cargar el catálogo inicial desde un archivo CSV
`GET /api/parametrizacion/auditoria`	Administrador	Consultar la auditoría de parametrización
`GET /health`	Público	Estado de la plataforma y de cada servicio

11.2 Cómo levantar la plataforma

Comando	Qué hace
`cp .env.example .env`	Crea el archivo de configuración local
`docker compose up --build`	Levanta los cuatro contenedores
`make seed`	Carga dos sedes, veinte mesas, el catálogo y un usuario por perfil
`make test` · `make cov`	Corre las 292 pruebas, con o sin informe de cobertura
`make evidencias`	Genera la evidencia de la Sprint Review en `docs/evidencias/`

11.3 Repositorio documental

Todo el material del proyecto vive en el repositorio `github.com/Juanse985/BAR-DE-MOE`:

- `docs/` — los documentos 01 a 08 listados en la sección 1.4, más este documento de arquitectura.
- `docs/evidencias/` — reporte de pruebas, prueba de humo, medición del tiempo de respuesta y capturas del frontend.
- `docs/postman/` — colección de Postman con las 27 peticiones de la API.
- `SPRINT 1/` (aula) — carta de presentación, propuesta comercial e historias de usuario.